Towards Agent-Based Explanations for Link Prediction

Bachelor Thesis

Nisa Thomas
University of Geneva

Graph Neural Networks (GNNs) excel at link prediction tasks but remain opaque, limiting their use in high-stakes domains. In this exploratory study, we adapt AgentNet, an inherently interpretable agent-based GNN, to the link prediction setting with just two simple changes: splitting agent initialization between source and target nodes and swapping in a binary readout head. On the dense Karate Club and the sparse GEPhilNet graphs, Link-AgentNet converges in nine and three traversal steps respectively, achieving decent accuracy and F1 scores without extensive hyperparameter tuning. Crucially, single-node-ablation experiments suggest that on the small dense graph, agents focus on a small set of 'make or break' nodes, while in the hub-and-spoke graph, they flexibly reroute among multiple hubs. Our findings suggest that built-in path-based explanations can coexist with solid predictive performance, pointing toward more transparent link-prediction GNN designs.

1 Introduction

1.1 Context

The relationship between entities across domains can be represented as a network, typically modeled as a graph. Link prediction is a fundamental task that aims to predict the formation of future edges between these entities from known data. It finds application in diverse fields including new relationship recommendations in social networks (Song et al., 2020), identifying interactions between proteins (Zhou et al., 2020) or even inferring connections in knowledge graphs (Zhao et al., 2023).

Traditional methods used heuristic scores (e.g., Common Neighbors, Preferential Attachment, Katz). They rely on strong assumptions that limit their ability to generalize across different types of graphs (e.g., social vs. biological networks). In fact, heuristics can be viewed as predefined graph structure features. Instead of defining features that stem from assumptions, a more natural way would be to learn the features (Zhang and Chen, 2018). Graph Neural Networks (GNNs) emerged as a powerful paradigm for learning relational features automatically through recursive neighbor aggregation and node embedding updates (Thekumparampil et al., 2018).

1.2 Explainability Gap

GNNs demonstrate remarkable success in various tasks such as node classification and link prediction in different domains, but their application in sensitive domains such as criminal justice (Agarwal, Lakkaraju, and Žitnik, 2021), drug discovery and finance (Zhao, Kang, and Cheng, 2024) remains limited due to their opacity (Kakkad et al., 2023).

Indeed, GNNs are complex non-linear models, as they operate by combining feature information with combinatorial graph structures, which makes understanding the underlying factors behind their predictions all the more challenging. In light of this, many have focused on developing methods for explaining

GNN predictions. Methods broadly fall into *post-hoc* explainers (e.g. perturbation or gradient methods applied after training) and *ante-hoc* or self-explainable GNNs (which learn their own explanations) (Kakkad et al., 2023). Unfortunately, post-hoc methods are vulnerable to adversarial attacks (Li et al., 2024), and many SE-GNNs only guarantee faithfulness to their learned explanation objects (e.g., prototypes), not to the actual nodes and edges, because mapping those objects back to nodes, edges and/or features is only approximate (Christiansen et al., 2023).

These shortcomings leave two open questions unaddressed:

- Can we design a self-explainable model whose explanation is itself part of the forward computation, so that no "object-to-node/edges" mapping is required ?
- Can such a model be made to work for link prediction, a task for which very few SE-GNNs or post-hoc explainers provide instance-level rationales ?

1.3 Proposed Solution & Contributions

AgentNet (Martinkus et al., 2022) offers a promising starting point. Instead of the traditional message-passing approach that standard GNNs employ, messages flow through agents that intelligently traverse the graph for a certain number of steps, after which the decision on the specific task is made collectively. Consequently, the sequence of visited nodes serves as the inherent explanation for the model's decision.

The original architecture was developed for graph-level tasks. Hence, its suitability for link prediction and the faithfulness of its trajectories in that setting remain open. In this thesis, we aim to address this gap by answering the following questions:

1. **RQ1**: Can the AgentNet framework be adapted to link prediction while preserving its native path-level explanations?
2. **RQ2**: How does the adapted AgentNet perform on graphs with contrasting topologies (dense vs. sparse)?
3. **RQ3**: On identifying the most influential nodes on the trajectories, are the frequency of node visits correlated with the drop in logit when those nodes are ablated?

Rather than aiming for exhaustive benchmarking, this thesis takes an exploratory approach focused on inquiring the interpretability potential of AgentNet in the context of link prediction. Accordingly, we deliberately restrict our evaluation to a small scale, as the goal is not to provide a comprehensive validation of the proposed *Link-AgentNet*, but to explore its feasibility and generate insights for future research.

1.4 Thesis Roadmap

In the following sections, we first review the core principles of GNNs and their limitations, before examining GNN-based link-prediction techniques and existing explainer methods, highlighting the gaps that AgentNet is designed to fill. We then present the original AgentNet architecture and introduce our proposed extension for link prediction. Afterward, we describe our methodology, detail the experimental setup used to address our research questions, and report the resulting findings. Finally, we discuss these results, acknowledge the limitations of our approach, and suggest avenues for future research.

2 Theoretical Background and Literature Review

2.1 General Framework of GNNs

Graph Neural Networks are neural networks specifically designed for data structured as graphs. They can learn node, edge and graph level representations (Zheng, Lü, and Wei, 2024) that can be used for downstream tasks such as node classification, link prediction, graph classification and clustering (Zhou et al., 2020)(Dwivedi et al., 2020). These representations are learned through "message passing" (Gilmer et al., 2017), wherein vector messages flow between a graph's nodes along its edges. GNNs iteratively build each embedding by aggregating and merging features from its neighbors across successive layers. Each such layer corresponds to one hop of information propagation through the network (Zhou et al., 2020).

While there are a variety of GNN architectures, most conform to the same AGGREGATE-UPDATE transformations at each layer. Let $G = (V, E)$ be the input graph, h_u^k denotes the embedding of node $u \in V$ at iteration k, with h_u^0 the starting state of node u. At each layer, node u's embedding is updated as follows:

$$h_u^{k+1} = \mathrm{UPDATE}\left(h_u^k, \mathrm{AGGREGATE}\left(\{h_v^k \mid v \in \mathcal{N}(u)\}\right)\right)$$

Here, AGGREGATE gathers the previous-layer embeddings of u's neighbors ($\mathcal{N}(u)$) and UPDATE combines that aggregated message with u's previous-layer state.

The specific choice of functions for AGGREGATE and UPDATE is what differentiates GNN architectures. For example, GCN (Kipf and Welling, 2016a) uses a spectral-based aggregation function derived from graph convolution in the Fourier domain. In contrast, GraphSAGE (Hamilton, Ying, and Leskovec, 2017) and DCNNs (Atwood and Towsley, 2015) define these functions in the spatial domain, enabling message passing

based on local neighborhoods. GATs (Veličković et al., 2017) introduce an attention-based aggregation function that adaptively weights each neighbor's contribution. Other methods include Gated GNNs (Li et al., 2015), which use recurrence-based aggregators (LSTM, GRU). These variations in functional design give rise to the diverse landscape of GNN models(Zhou et al., 2020).

2.2 Limitations of GNNs

Despite their success, standard message-passing architectures exhibit several well-studied shortcomings :

- **Explainability gap** : The message-passing framework obscures the contribution of individual nodes and edges to a specific prediction, making it difficult to trace which parts of the graph influenced the model's decision (Kakkad et al., 2023).
- **Oversmoothing & oversquashing** : As GNN layers increase, node embeddings tend to become indistinguishable (oversmoothing) (Oono and Suzuki, 2019) or compress too much information (oversquashing) (Alon and Yahav, 2020).
- **Indirect/redundant flow** : Message-passing cannot selectively ignore irrelevant parts of the graph. Every node indirectly contributes to every other through neighborhood aggregation. Attention mechanisms (e.g., GAT) were introduced to softly control what information is aggregated, but they still aggregate over entire neighborhoods, making noisy information difficult to filter out (Martinkus et al., 2022).
- **Scalability bottlenecks** : Most expressive GNNs cannot scale to graphs with thousands of nodes due to exponential aggregation costs (Martinkus et al., 2022).
- **Task generalization challenges** : Many expressive GNN variants are optimized for graph-level tasks, and may not transfer well to node-level or link prediction tasks, nor generalize across graph domains (Tola, Myrick, and Coskunuzer, 2024).

In contrast, AgentNet evades many of these issues by replacing global neighborhood aggregation with learned agent traversals:

- **Built-in interpretability**: each agent's discrete path is itself the explanation.
- **Mitigated oversmoothing & oversquashing**: because an agent only visits one node per step, embeddings do not collapse or aggregate information exponentially with the number of steps (Martinkus et al., 2022).
- **Scalability**: the model's complexity is graph size-independent. It only depends on the number of agents, the number of steps and node degree.
- **Generalizability**: As we show in the results, AgentNet performs decently on link prediction tasks with minimal modifications.

2.3 GNNs for Link Prediction

GNN-based link prediction methods generally fall into two main paradigms. The first is node embedding-based approaches, such as GAE and VGAE (Kipf and Welling, 2016b), which use a GCN encoder to learn node representations and then apply a decoder (typically a dot product) to estimate the likelihood of a link between node pairs. VGAE extends GAE by modeling each node's embedding as a distribution rather than a fixed vector. The second paradigm is subgraph-based approaches, popularized by SEAL (Zhang and Chen, 2018), which extract an enclosing subgraph around a candidate link and feed it into a GNN for classification. The prediction is based on both the subgraph structure and a node information matrix that captures relevant node features. Variants of both paradigms exist, including GraphSAGE-based encoders (Ziya and Kumar, 2024) and attention-based subgraph classifiers (Lai et al., 2021), among others.

While effective, both methods inherit the aforementioned limitations of standard GNNs. Node-based learn node embeddings through recursive neighborhood embedding aggregation, which is not inherently interpretable. While the SEAL method is able to restrict the context of the target link to a k-hop neighborhood around it, it still remains opaque. The node information matrix in SEAL includes structural labels (DRNL), possibly original node attributes and implicit positional encoding learned by the GNN during training. These combined sources of information are then passed through message-passing layers, further obscuring the rationale that drives the prediction.

2.4 GNN Explainability

(Kakkad et al., 2023) organize existing explainers along two axes :

1. *When* they explain:
    - **Post-hoc**: Explanations are derived post-training using perturbation or gradient-based methods, surrogate models, etc.
    - **Ante-hoc** (SE-GNNs): Models jointly learn a predictor and a subgraph/feature extractor during training.
2. *How* they explain:
    - **Factual**: Looks for the nodes/edges/features that contribute most to the original prediction.
    - **Counterfactual**: Looks for the minimal change that flips the prediction.

Among these, ablation-based (or "perturbation") methods are a popular post-hoc explainability technique. They iteratively mask nodes or edges and rerun the model to observe changes in the output. We adopt this strategy to quantify the importance of nodes visited in our agents' trajectories.

Some representative explainers include GNNExplainer (Ying et al., 2019), which learns a soft mask over edges and node features by optimizing the prediction fidelity, and PGExplainer (Luo et al., 2020) which trains a small model to predict edge masks conditioned on node embeddings. Both are powerful, but as (Li et al., 2024) demonstrates, their learned masks may not faithfully reflect the GNN's true reasoning.

In contrast, AgentNet's forward pass *is* an explanation: its agents traverse specific nodes in sequence, and those traversal paths are the direct "highways" along which information flows. There is no separate mask to learn or concept-to-node mapping to perform, which sidesteps the main pitfalls of both post-hoc and many ante-hoc explainers.

3 AgentNet Model & Adaptation for Link Prediction

The limitations of the standard message-passing framework, combined with the shortcomings of existing explainers, highlight a core issue: the parallel neighborhood aggregation scheme used in message-passing GNNs was never intended to support interpretability. This calls for a paradigm shift from layer-wise message passing to more targeted, sequential reasoning over graph structure.

In the next section, we describe AgentNet (Martinkus et al., 2022), an agent-based architecture that replaces recursive neighborhood aggregation with learned traversals, offering step-by-step, sample-level rationales by design. We detail its components and describe the minimal modifications required to adapt it for link prediction.

3.1 AgentNet High-Level Workflow

Each agent is uniquely identifiable and initialized at a node position, either randomly or with a learned importance score. At time-step *t*, four operations are performed : node update, neighborhood aggregation, agent update and agent transition:

- **Node update**: Every occupied node is updated via a learnable function (typically an MLP) that takes as input the node's state at $t-1$ and the current states of all agents occupying it. To ensure expressiveness, the agent embeddings are first transformed through a shared learnable layer before being integrated into the node update.

$$v_i^t = f_v\left(v_i^{t-1}, \sum_{a_j^{t-1}\in\mathcal{A}(v_i)} \phi_{a\to v}(a_j^{t-1})\right)$$

Legend: v_i^t: embedding of node v_i at time t; $f_v(\cdot)$: node update function; $\mathcal{A}(v_i)$: agents at node v_i at time t; a_j^{t-1}: embedding of agent a_j at $t-1$; $\phi_{a\to v}(\cdot)$: learnable transformation of agent states.

- **Neighborhood aggregation step** : The embedding of each occupied node is updated by aggregating the embeddings of its neighbors $N(v_i)$. To improve expressiveness, each neighbor's embedding is first transformed by a learnable function.

$$v_i^t = f_n\left(v_i^{t-1}, \sum_{v_j\in N(v_i)} \phi_{N(v)\to v}(v_j^{t-1})\right)$$

Legend: $N(v_i)$: set of neighboring nodes of node i; $v_j^{t-1} \in N(v_i)$: embedding of neighbor node j at time $t-1$; $\phi_{N(v)\to v}(\cdot)$: learnable transformation applied to each neighbor; $f_n(\cdot,\cdot)$: node update function that combines v_i^{t-1} with the aggregated neighbor information.

- **Agent Update** : Each agent's state is then updated based on the node it is currently occupying.

$$a_i^t = f_a\left(a_i^{t-1},\ v_{V(a_i)}^t\right)$$

Legend: a_i^t: embedding of agent a_i at time step t; $f_a(\cdot)$: agent update function (e.g., an MLP); a_i^{t-1}: embedding of agent a_i at time $t-1$; $v_{V(a_i)}^t$: embedding of the node occupied by agent a_i at time t.

- **Agent Transition** : Next, each agent decides where to move by first identifying the set of possible next positions (i.e., the neighbors of its currently occupied node and the current node itself if self-loops are enabled).

$$N_{(a_i)}^t = N\left(v_{V(a_i)}^t\right) \cup \left\{v_{V(a_i)}^t\right\}$$

Legend: $N_{(a_i)}^t$: candidate set of next nodes agent a_i can move to at time step t; $v_{V(a_i)}^t$: node currently occupied by agent a_i at time t; $N(\cdot)$: neighborhood function returning adjacent nodes; the union includes the current node itself if self-loops are enabled.

For each possible move, the agent computes an attention score using its own state and the state of the potential node it wants to visit.

$$z_{a_i\to v_j} = f_p\left(a_i^t,\ v_i^t\right)$$

Legend: $z_{a_i\to v_j}$: attention score for agent a_i moving to node v_j; $f_p(\cdot)$: scoring function; a_i^t: embedding of agent a_i at time t; v_i^t: embedding of the current node occupied by agent a_i at time t.

Finally, the logits are passed to a Gumbel Softmax function in order to select a discrete action.

This sequential and localized agent movement allows for a selective aggregation of information relevant to the task, which in turn allows for more interpretable results.

3.2 Implementation Details: Parameter Settings

In its implementation[1], the authors of AgentNet offer a diverse range of configurable behaviors that allow for both architectural flexibility and exploratory research. These hyperparameters control the agents' movement strategies, an optional global information-sharing mechanism and the model's readout configuration which we detail below.

3.2.1 Agent Movement

The model supports three movement modes. In random mode (`random_agent = True`), agents simply pick a neighboring node at random. You can switch off randomness (`random_agent = False`) to use the attention-based mode, where a layer scores each neighbor and the agent moves to the highest-scoring node.

On top of attention, a heuristic-based mode can be enabled. When `basic_global_agent = True`, all agents share a single set of learnable scalar scores. In contrast, `basic_agent = True` assigns each agent its own MLP to learn these scores. If `bias_attention = True`, these heuristic scores are added to the attention scores, gently biasing the agent's movement without overriding the learned policy.

There are four such heuristic parameters:

- `explored_param` raises the score of already-visited nodes
- `unexplored_param` boosts never-visited nodes
- `stay_param` favors staying in place (when self-loops are allowed)
- `back_param` rewards going back to the immediately previous node

3.2.2 Global Context Integration (Virtual Node Analog)

The model also offers an analogue to virtual nodes via two flags. If you set `global_agent_pool = True`, each node update is enriched by the average of *all* agent embeddings at that step. Turning on `agent_global_extra = True` then concatenates that same global summary to every agent's state before it's updated, giving each agent a view of the group's overall behavior.

[1] `https://github.com/KarolisMart/AgentNet/blob/main/model.py`

3.2.3 Readout Options

Readout happens after traversal and can be configured in several ways. If `final_readout_only = True`, only the last-step agent embeddings are fed to the prediction head; otherwise, the predictions are averaged from every step. Enabling `node_readout = True` adds the global mean-pooled node embedding to the agent-level readout. For even finer control, `use_step_readout_lin`, `step_readout_mlp`, `readout_mlp` and `mean_pool_only` let you insert linear layers or MLPs before or after pooling, and choose between mean-only versus combined mean-and-max aggregation.

3.3 Modifications for Link Prediction

We adapt the AgentNet model for link prediction by adding two minimal modifications (Figure 1):

- **Agents' initial positions**: the agent traversal is now conditioned directly on the target link. Half of the agents' positions are initialized at the source node and the other half at the target node. These agents then explore the surrounding neighborhood for a fixed number of steps, following the original AgentNet forward-pass flow (i.e., node update, neighborhood aggregation, agent update and agent transition).
- **Prediction**: After the traversal, the final agent states (and optionally the updated node states) are aggregated and passed to a binary classification head, which outputs the predicted logits for link existence between the source and target nodes.

With these adaptations, AgentNet can be applied to link prediction without altering its core traversal logic, thereby preserving its inherent interpretability: each agent's path remains a transparent, sample-level explanation. In the next chapter, we present our experimental setup, evaluate Link-AgentNet on two graphs with distinct topologies, and explore whether node visitation frequency correlates with prediction importance.

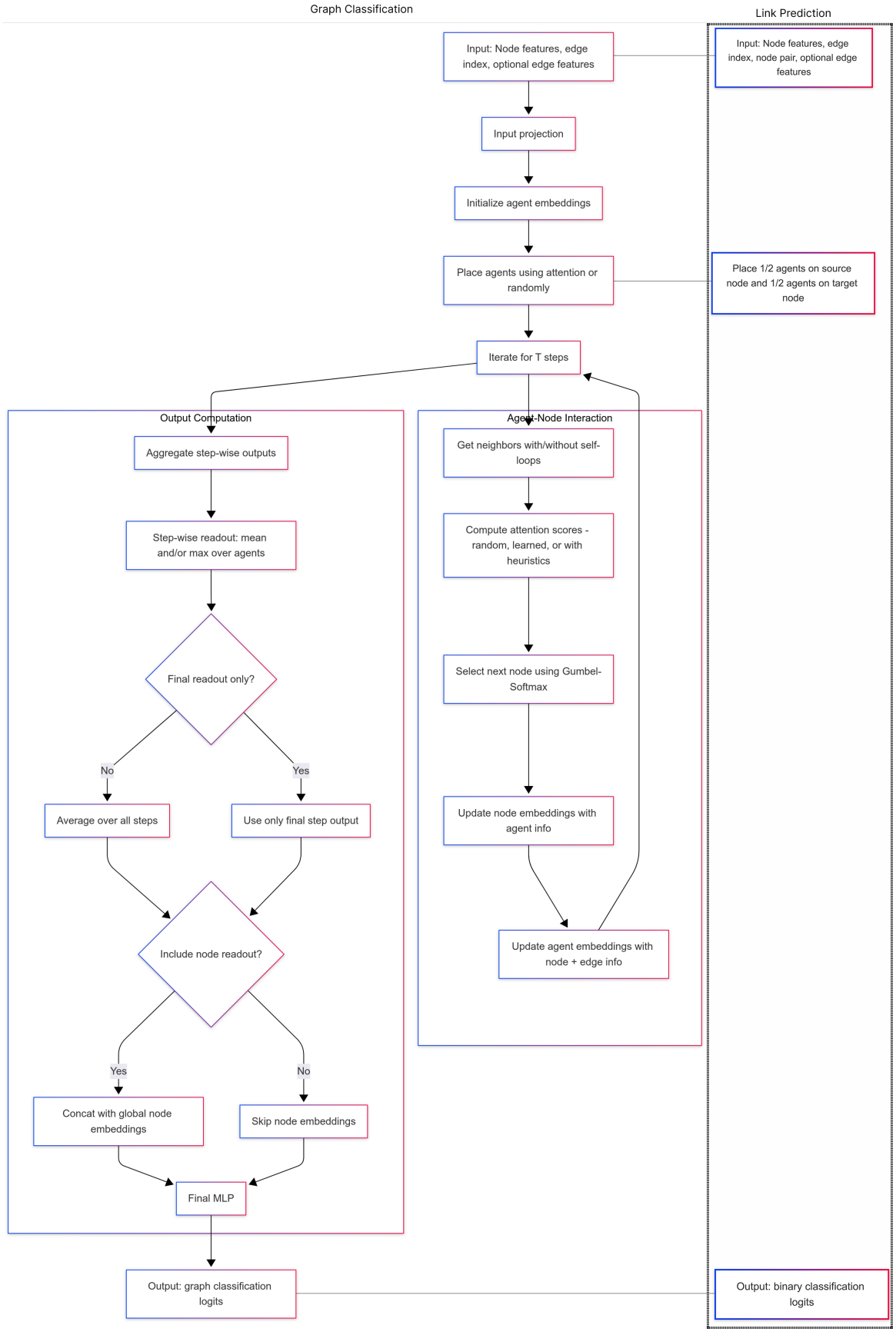

Figure 1: *AgentNet forward-pass flow diagram. Original components (left) are visually paralleled with their modified counterparts (right) to highlight architectural changes made for link prediction.*

4 Methodology

4.1 Dataset Preparation

4.1.1 Karate Club Dataset

We use the well-known Karate Club dataset provided by the PyTorch Geometric library, which consists of 34 nodes representing members of a university karate club and 78 edges representing social interactions between the members outside the club.

As the dataset does not come with node features, we generate basic structural features such as node degree, clustering coefficient, betweenness centrality and PageRank score. Additionally, we compute community assignments using the greedy modularity optimization algorithm, and encode these into one-hot vectors. The final node feature is the vector concatenation of the normalized structural features and the community one-hot encoding.

4.1.2 Geneva's Philanthropic Network (GEPhilNet)

The GEPhilNet dataset is a custom-built graph representing geneva-based philanthropic foundations and their relations. There are 128 nodes represent foundations, and 202 edges represent relationships based on either shared board members and/or financial transactions.

To construct this graph, financial transaction data was obtained in collaboration with *Fundraiso*, a platform that maintains a directory of philanthropic organizations along with details such as their purpose, metadata, board members, and financial transactions when available. Board membership data was collected from the Geneva canton's commercial registry. Since financial disclosure on *Fundraiso* is voluntary, not all Geneva-based foundations listed in the cantonal registries report their financial transactions on the platform. Consequently, the GEPhilNet graph is limited to the subset of foundations that have made such disclosures.

Similarly to Karate Club, we used structural features such as degree, clustering coefficient, betweenness centrality and PageRank, as well as community membership via modularity-based clustering to construct node features. We did not use foundation-specific features (e.g., funding amounts, mission statements, support areas) due to incomplete or missing data across many nodes. Using them would have significantly reduced the usable graph size.

In addition to node features, we also add edge features such as the number of financial transactions recorded between the two foundations and the number of shared board members between them.

4.1.3 Link Prediction Dataset Construction

To construct supervised link prediction datasets from both graphs, we implemented a custom LinkPrediction-Dataset class in PyTorch. The key steps are:

- Positive edges are taken directly from the edge list of the graph. For GEPhilNet, we include only positive edges where the transaction count between two foundations is greater than zero.
- Negative edges are uniformly sampled from non-connected node pairs until we get the same number of negative and positive samples.
- For each sample, (positive or negative), we extract a k-hop enclosing subgraph around the target link (with $k = 3$)
- Direct edges between the positive node pair's are removed from the subgraph to avoid label leakage.

Each sample is encoded as a PyG Data object, including:

- `x` : node features of the subgraph
- `edge_index` : adjacency of the subgraph
- `edge_attr` : edge attributes if present
- `y` : binary label (0 or 1)
- `node_pair` : the source and target node IDs in the subgraph
- `batch` : batch index

We skip samples where the removal of a positive edge in its corresponding subgraph causes either the source or target node to become isolated. We also skip the positive samples whose removal causes a disconnected subgraph.

4.2 Training and Model Evaluation

The training and evaluation pipeline builds directly on the authors' original AgentNet codebase[2] and retains its stratified cross-validation setup. To accommodate link prediction, we replace the multiclass objective for a binary cross-entropy loss on the raw logits $\hat{y}_i$ for each node pair:

$$L_{\mathrm{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log\left(\sigma(\hat{y}_i)\right) + (1-y_i)\log\left(1-\sigma(\hat{y}_i)\right)\right]$$

Legend: L_{BCE}: binary cross-entropy loss; N: total number of samples; $y_i \in \{0,1\}$: ground-truth label for the i-th sample; $\hat{y}_i$: predicted logit; $\sigma(\cdot)$: sigmoid activation function.

After each epoch, the model is evaluated on the test set using accuracy, area under the ROC curve (AUC) and F1 Score. The best epoch is selected based on the highest mean F1 score over the test set across all splits.

[2] `https://github.com/KarolisMart/AgentNet/blob/main/graph_classification.py`

4.3 Hyperparameter Optimization

Hyperparameter tuning is performed using Optuna (Akiba et al., 2019), with the objective of maximizing the average F1 score over the test splits of all cross-validation folds. Due to the great number of hyperparameters, only certain key hyperparameters are used in the search space:

Parameter	Values Searched
Threshold for classification	`[0.1, 0.9]`
Number of agents	`{4, 6}`
Batch size	`{16, 32}`
Hidden units	`{8, 16, 32}`
Number of traversal steps	`{3, 6, 9}`
Learning rate (log-uniform)	`[1e-4, 1e-2]`
Dropout	`[0.0, 0.5]`
Readout type	`{final_only, all_steps, all_steps_linear, 'all_steps_mlp}`
Attention heads	`{1, 2, 4}`
Gumbel temperature	`[0.3, 1.0]`

Table 1: *Key hyperparameters searched using Optuna and their corresponding value ranges.*

While all other parameters are kept at their default values, we explicitly enable `global_agent_pool` and `agent_global_extra` to provide agents with access to a shared global embedding. We also enable `basic_global_agent`, `bias_attention` and `self_loops`, allowing agent movement to be influenced by both heuristic preferences and learned attention scores.

We capped the number of trials for Karate Club to 30 and 60 for GEPhilNet. Each trial runs all cross-validation splits for 200 epochs. Due to the small size of the datasets and time restrictions, we opted for $k = 2$ folds.

4.4 Explainability analysis of AgentNet trajectories

AgentNet is *self-explainable*: each prediction arises from agents that explicitly choose one edge to traverse at each step. Their trajectories therefore form a transparent, human-understandable rationale. However, we also conducted a lightweight post-hoc XAI analysis to quantify agent trajectories, examine how they differ from a random-walk baseline, and evaluate whether frequently visited nodes contribute meaningfully to the model's prediction. All measurements are computed per test sample and later aggregated over splits.

4.4.1 Trajectory extraction

For each dataset and its corresponding test splits, we reload the checkpoint with the best mean F1 score for that split and evaluate two model variants on each test sample:

Model	Policy Flag
Learned	`random_agent = False`
Random-walk	`random_agent = True`

Table 2: *Policy flag settings used to configure learned vs. random-walk agent behavior.*

During inference we set the number of batches to 1 so that agent trajectories can be captured per sample. We have modified the forward pass to also return paths in the form of:

$$\text{paths} = \left\{ t \mapsto \mathbf{a}^{(t)} \right\}_{t=0}^{T}$$

Legend: paths: full agent trajectory history; $\mathbf{a}^{(t)} \in \mathbb{N}^A$: vector of node IDs occupied by the A agents at step t; $t = 0$: initial step where agents begin from their assigned start nodes.

4.4.2 Visitation frequency and energy

For each sample, we build a local frequency distribution:

$$p(i) = \frac{\#\text{ visits to node } i}{A \cdot T}, \quad i \in \{0, \ldots, N-1\}$$

Legend: $p(i)$: visitation probability of node i; A: number of agents; T: number of steps per agent; $\#$ visits to node i: total number of visits to node i across all agents and time steps.

ignoring the source and target nodes of the node pair.

We can also define node-level visitation energy such as $E(i) = p(i)^2$, which squares down the weights of rarely visited nodes and amplifies the frequently visited ones.

4.4.3 Path divergence

In order to quantify the difference between the sets of visited nodes by agents using a learned policy versus a random-walk policy, we use the weighted Jaccard similarity, defined as :

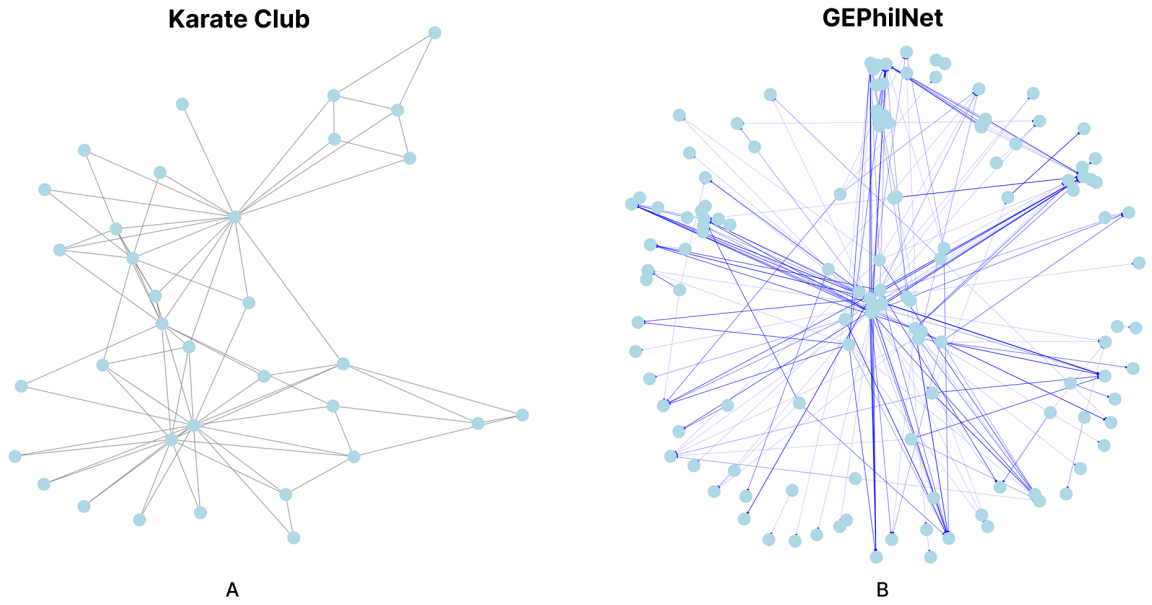

Figure 2: *A, B: Network plots of Karate Club and GEPhilNet with edge weight proportional to financial transaction frequency (spring layout); C: Table of summaries of both graphs*

$$J_w(p,q) = \frac{\sum_i \min\{p(i),\ q(i)\}}{\sum_i \max\{p(i),\ q(i)\}}$$

Legend: $J_w(p,q)$: weighted Jaccard similarity between two visitation distributions; $p(i)$ and $q(i)$: visitation frequencies of node i under two different policies; ranges between $[0,1]$, with the numerator summing shared visitation mass and the denominator summing total mass across both.

4.4.4 Node importance via single-node ablation

For each visited node k, we zero its feature and recompute the logit. The difference between the new logit and its original logit prior to ablation is noted as :

$$\triangle_k = \left|\hat{y} - \hat{y}_{\setminus k}\right|$$

Legend: $\triangle_k$: absolute change in predicted logit after ablating node k; $\hat{y}$: original prediction logit; $\hat{y}_{\setminus k}$: prediction logit after setting node k's embedding to zero.

A high $\triangle k$ would indicate that node k is causally important for the prediction.

4.4.5 Energy - Importance correlation

To test whether agents actually prioritize influential nodes, we compute, on the subset of visited nodes, the Spearman rank correlation:

$$\rho = \text{corr}_{\text{Spearman}}(E, \triangle)$$

Legend: E: node visitation energy; $\triangle$: logit difference after node ablation

For both learned and random-walk policies. Samples with < 2 unique values in either vector are assigned $\rho = \text{NaN}$.

4.4.6 Performance deltas and statistical tests

Per split of each dataset, we store accuracy, AUC and F1 score for both policies and report their differences $\triangle\text{metric} = \text{learned} - \text{RW}$. Significance of $\triangle$ values is assessed with a two-sided paired Wilcoxon test.

5 Results

5.1 Dataset Characteristics

Karate Club is undirected (34 nodes, 78 edges), whereas GEPhilNet is a directed real-world graph (128 nodes, 202 edges). As Table 3 shows, Karate Club is almost 12x more denser (0.0139 vs 0.012) and shows strong triadic closure (clustering $= 0.57$) in comparison to GEPhilNet. We can observe these through the network plots as well (Figure 2), where GEPhilNet exhibits a star-shaped topology with edge "super-hubs" at the center, while Karate Club displays a more balanced, community-like structure, with two main clusters connected by a few bridge nodes. The degree distribution's skewness implies a heavy-tailed distribution in GEPhilNet, suggesting that only a few philanthropic actors mediate most transactions.

Figure 3: *A, B: Achieved objective values per trial through Optuna hyperparameter tuning for Karate Club and GEPhiNet.*

Metric	Karate Club	GEPhilNet
Directed	No	Yes
# Nodes	34	128
# Edges	78	202
Density	0.139	0.012
Clustering Coefficient	0.57	0.0036
Median Degree	3	2
Degree Dist. – 25th/75th percentiles	`[2, 5]`	`[1, 4]`
Degree Dist. – Skewness	2.00	2.67

Table 3: *Structural statistics of the Karate Club and GEPhilNet graphs.*

5.2 Hyperparameter search

Hyperparameter	Karate Club	GEPhilNet
Classification Threshold	0.44	0.25
# Agents	6	6
Batch size	16	16
Hidden units	32	32
# Traversal steps	9	3
Learning rate	0.0013	0.00058
Dropout	0.20	0.021
Readout type	`all_steps`	`all_steps_linear`
Attention heads	2	1
Gumbel temperature	0.46	0.47

Table 4: *Best hyperparameters found by Optuna for Karate Club and GEPhilNet.*

Figure 3 shows that over 30 Optuna trials, the objective on the Karate Club graph climbs steadily from about 0.75 to roughly 0.85, with noticeable upward jumps, whereas on GEPhilNet, it stabilizes near 0.73 after just a few trials due to faster convergence. We limited Karate Club to 30 trials for time reasons; we would need more trials to observe a plateau.

The best Optuna-found hyperparameters are displayed in Table 4. It shows nine traversal steps as optimal for the dense Karate Club but only three for the sparser GEPhilNet. We hypothesize that the abundance of closed triangles in Karate Club requires more traversal steps for agents to effectively explore these motifs. GEPhilNet's hub-and-spoke structure, by contrast, could provide direct "highways" between clusters, allowing agents to cover the graph effectively in fewer moves.

Another notable difference is the dropout values (0.20 vs 0.021). The lower dropout value for GEPhilNet might be to combat underfitting in sparse data. We also observe a lower threshold value for GEPhilNet (0.25 vs. 0.44 for Karate Club), which suggests reduced confidence for the prediction of positive links. Conversely, Karate Club has stronger positive-class signals. Its connectivity patterns might give the model higher confidence in assigning high logits to real edges.

The disparity in optimal hyperparameters already hints that learned traversal policies adapt to underlying graph topology.

5.3 Predictive Performance

Figure 4 compares training dynamics under the best Optuna-selected hyperparameters:

- **GEPhilNet** converges quickly: both training and validation losses flatten by epoch 25 with no sign of

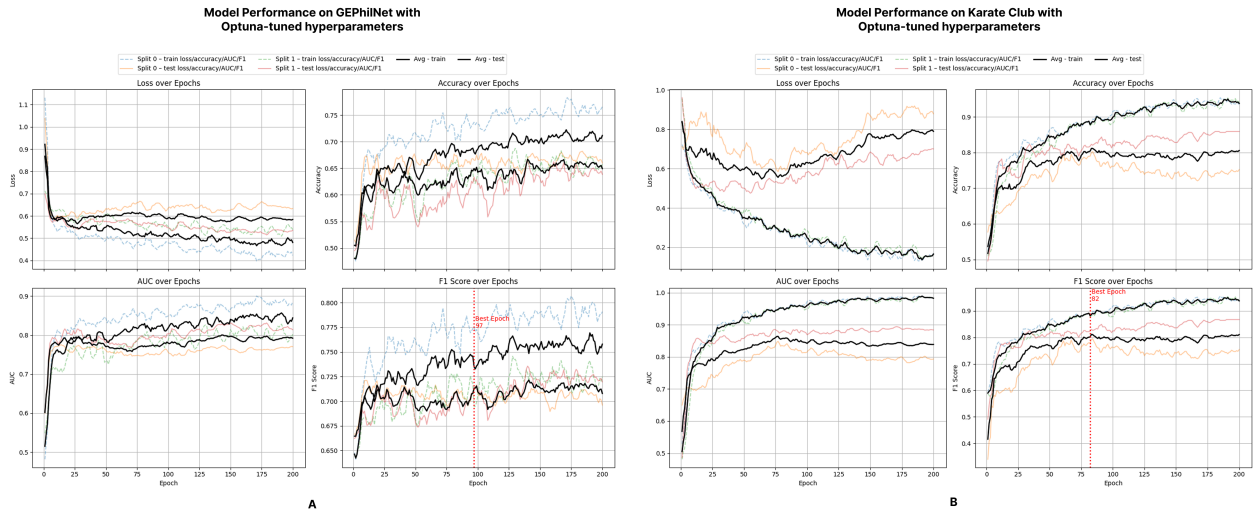

Figure 4: *A, B: Model performance plots (Loss, Accuracy, AUC, F1) on training and test splits of GEPhilNet and Karate Club, with their respective best Optuna-found hyperparameters.*

overfitting as training continues. Accuracy levels off around 0.70 on the training splits and 0.65 on the test splits, reflecting the dataset's complexity.

- **Karate Club** shows a steady decline in both losses until roughly epoch 80, after which training loss continues to drop while test loss rises, which is evidence of overfitting. Nevertheless, it reaches high accuracy (≈ 0.9 on training, ≈ 0.8 on test) at its optimal epoch. Its AUC plateaus near 0.85 on the test splits, indicating very strong ranking performance.

When we select the final models based on mean F1 score across both splits, Karate Club achieves an F1 score ≈ 0.8, whereas GEPhilNet reaches ≈ 0.71, again underscoring the greater challenge posed by the sparser, more complex GEPhilNet graph.

While the absolute scores differ, both sets exhibit consistent train-test trends relative to their complexities, indicating that the performance variations are heavily impacted by factors that are intrinsic to the datasets rather than the hyperparameters themselves. In the next section, we shall analyze how agents' movement policies differ with each dataset.

5.4 Explainability

5.4.1 KarateClub (undirected, 34 nodes)

The learned policy walks deviate strongly from random walks (only 22 % weighted path overlap) and this divergence yields a statistically significant $+1$ pp accuracy and $+0.9$ pp F1, with a tiny AUC drop. Visitation energy is modestly correlated with single-node importance ($\rho \approx 0.10$), which is greater than the random baseline by $+0.03$.

Policy	Acc	F1	AUC	J_W	ρ
Learned	0.822 ± 0.005	0.837 ± 0.007	0.897 ± 0.010	0.22 ± 0.11	0.098 ± 0.35
Random-walk	0.812 ± 0.010	0.829 ± 0.0005	0.899 ± 0.014	—	0.07 ± 0.37
Δ	**+0.010**	**+0.009**	**–0.003**	—	**+0.03**
p-value	2.9×10^{-19}	2.5×10^{-72}	2.9×10^{-19}	—	—

Table 5: *Performance and explainability metrics for Karate Club under learned and random-walk policies. Δ refers to the absolute difference (Learned – Random), and p denotes Wilcoxon signed-rank test results.*

5.4.2 GePhilNet (directed, 128 nodes)

Policy	Acc	F1	AUC	J_W	ρ
Learned	0.712 ± 0.084	0.762 ± 0.038	0.837 ± 0.024	0.34 ± 0.09	0.14 ± 0.25
Random-walk	0.705 ± 0.064	0.742 ± 0.022	0.840 ± 0.015	—	0.17 ± 0.24
Δ	**+0.006**	**+0.020**	**–0.003**	—	**–0.03**
p-value	2.7×10^{-8}	4.8×10^{-29}	2.7×10^{-8}	—	—

Table 6: *Performance and explainability metrics for GEPhilNet under learned and random-walk policies. Δ refers to the absolute difference (Learned – Random), and p denotes Wilcoxon signed-rank test results.*

Path overlap is larger (34 % overlap), yet the F1 score of the learned-policy is higher by 2 pp. We have a slightly higher energy-importance correlation than Karate Club but the learned policy's correlation trails the random-walk by about 0.03.

For both datasets, the learned policy only edges out the random walk by a few tenths of a percent ($+0.9$ pp in accuracy and F1 on Karate Club, $+0.7$ pp accuracy and $+2.0$ pp F1 on GEPhilNet), but these gains are highly significant (p $\ll 0.001$). It achieves them by redirecting roughly 70-80% of their visitation probability to a different subset of nodes. In the Karate Club dataset, these rerouted visits align more closely with ablation-derived importance scores than those of the random-walk baseline.

Metric	Karate Club	GEPhilNet
Nodes / Edges	34 / 78	128 / 202
Density	0.139	0.012
Clustering	0.57	0.004
Accuracy ($\mu \pm \sigma$)	0.82 ± 0.01	0.71 ± 0.02
F1 ($\mu \pm \sigma$)	0.84 ± 0.01	0.76 ± 0.02
AUC ($\mu \pm \sigma$)	0.897 ± 0.010	0.837 ± 0.024

Table 7: *Structural and performance metrics for Karate Club and GEPhilNet.*

6 Discussion

6.1 Extending AgentNet to link prediction and its limitations

We demonstrate that the original AgentNet framework can be extended for the link prediction task with minimal modifications. Specifically, only the initialization of agent positions and the output layer are adapted, as the latter is changed from a softmax to a binary logit. This approach allows us to extract the exact paths that drive a sample-level decision, providing a faithful step-by-step rationale for each predicted edge.

Regarding computational overhead, this approach for a per-edge explainability is not free. For every new node pair, we must (1) extract its k-hop subgraph and (2) run A agents for T steps. This can be expensive when wanting to process large amounts of node pairs. We could argue, however, that this trade-off is justified in high-stakes domains, where transparency is crucial.

A further limitation is that we must omit a handful of positive edges during training to avoid disconnected subgraphs. This also poses a challenge when predicting links for a new, unseen node that has no existing connections in the graph. A potential solution, inspired by (Hwang et al., 2022), is to add one or more virtual nodes to the graph that collect messages from all real nodes but never send messages back. This keeps every subgraph connected while letting agents use the virtual node purely as a transit hub. We leave empirical validation of this idea to future work.

6.2 Predictive performance and graph topology

We observe higher absolute scores on the dense graph: its small size and tightly clustered neighborhoods make link prediction comparatively easy, yielding Acc ≈ 0.82 and $F1 \approx 0.84$. On the sparse, hub-centric graph, performance is lower but still respectable, with F1 capped around 0.76 and AUC plateauing early. Having established the model's raw predictive power, we now turn from predictive accuracy to analyzing whether the frequency with which agents visit nodes aligns with those nodes' actual contribution to the prediction.

6.3 Explainability: learned vs random-walk trajectories

Dataset	J_W (Learned vs RW)	$\Delta\rho = \rho_L - \rho_{RW}$
Karate Club	0.22 ± 0.11	$+0.03$
GEPhilNet	0.34 ± 0.09	-0.03

Table 8: *Weighted Jaccard similarity between learned and random-walk trajectories, and difference in correlation ρ between node visitation energy and logit drop.*

For Karate Club, the fact that learned trajectories diverge strongly from its counterpart, and suffer a larger performance drop when their most-visited nodes are removed, suggests that the learned policy identified a small set of critical nodes in the graph and has learned to concentrate on them rather than exploring uniformly. In other words, the learned policy adapted itself to the graph's topology by zeroing in on the nodes that matter most. This is also reflected through the optimal threshold that settles around 0.44, suggesting that zeroing in on those critical connectors allows for greater confidence in predicting links.

In GEPhilNet, by contrast, learned trajectories are closer to random walks and are less affected by ablating their most-visited nodes than the random baseline. Since random walks inherently revisit high-degree nodes, the "super-hubs", removing those nodes hurts the random policy's logits more, which confirms their importance for confident link prediction. At the same time, the learned policy being less affected by the single-node ablations indicates that the learned policy allows for reroutes through alternative hubs, showing that GEPhilNet's link signal is distributed across multiple paths. It further suggests that the learned policy doesn't over-commit to one route when several equally informative routes exist, making it as comprehensive as a random walk but more robust to single-node failures. This raises a new question: how many distinct or overlapping sub-paths carry enough signal to yield the

same prediction? If dozens of detours are equally valid, then any one trajectory offers only a partial explanation. Quantifying how often agents switch to alternate paths under node ablation would reveal the graph's evidence redundancy, a topic we leave for future work.

6.3.1 Conclusion

Our experiments confirm that AgentNet can be ported to link prediction almost unchanged and still yields path-level rationales. On the small and highly-clustered Karate Club graph, the model reaches high absolute scores and its trajectories diverge strongly from a degree-biased walk while remaining causally aligned with the prediction ($\triangle\rho \approx +0.03$). On the sparser hub-and-spoke GEPhilNet, the learned policy still beats the random walk but seems to have the ability to adapt by flexibly rerouting around masked hubs, leading to a negative $\triangle\rho$ (-0.03).

This thesis could be further extended by empirically verifying the stated assumptions and possible remedies to the model's limitations. We could test whether a virtual-node extension could overcome the case of skipped positive edges whose removal disconnects the corresponding subgraphs. Furthermore, we could explore multi-node ablation as single-node ablations may be too simplistic to understand the interaction with logit difference. We could also explore if learned policies have a significant advantage over random policies in larger graphs ($> 10^k$ nodes). In addition, examining AgentNet's heuristic parameters could provide insight into whether different agents learn distinct traversal behaviors, especially in more complex graph structures. This could serve as a form of global explanation, complementing the local insights derived from individual agent trajectories.

Bibliography

Agarwal, Chirag, Himabindu Lakkaraju, and Marinka Žitnik (Jan. 2021). "Towards a Unified Framework for Fair and Stable Graph Representation Learning". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2102.13186. URL: https://arxiv.org/abs/2102.13186.

Akiba, Takuya et al. (Jan. 2019). "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1907.10902. URL: https://arxiv.org/abs/1907.10902.

Alon, Uri and Eran Yahav (Jan. 2020). "On the Bottleneck of Graph Neural Networks and its Practical Implications". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2006.05205. URL: https://arxiv.org/abs/2006.05205.

Atwood, James and Don Towsley (Jan. 2015). "Diffusion-Convolutional Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1511.02136. URL: https://arxiv.org/abs/1511.02136.

Christiansen, Marc et al. (Jan. 2023). "How Faithful are Self-Explainable GNNs?" In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2308.15096. URL: https://arxiv.org/abs/2308.15096.

Dwivedi, Vijay Prakash et al. (Jan. 2020). "Benchmarking Graph Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2003.00982. URL: https://arxiv.org/abs/2003.00982.

Gilmer, Justin et al. (Jan. 2017). "Neural Message Passing for Quantum Chemistry". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1704.01212. URL: https://arxiv.org/abs/1704.01212.

Hamilton, William L., Rex Ying, and Jure Leskovec (Jan. 2017). "Inductive Representation Learning on Large Graphs". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1706.02216. URL: https://arxiv.org/abs/1706.02216.

Hwang, EunJeong et al. (Jan. 2022). "An Analysis of Virtual Nodes in Graph Neural Networks for Link Prediction". In.

Kakkad, Jaykumar et al. (Jan. 2023). "A Survey on Explainability of Graph Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2306.01958. URL: https://arxiv.org/abs/2306.01958.

Kipf, Thomas and Max Welling (Jan. 2016a). "Semi-Supervised Classification with Graph Convolutional Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1609.02907. URL: https://arxiv.org/abs/1609.02907.

– (Jan. 2016b). "Variational Graph Auto-Encoders". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1611.07308. URL: https://arxiv.org/abs/1611.07308.

Lai, Darong et al. (Oct. 2021). "Attention Based Subgraph Classification for Link Prediction by Network Re-weighting". In: DOI: 10.1145/3459637.3482060. URL: https://doi.org/10.1145/3459637.3482060.

Li, Yujia et al. (Jan. 2015). "Gated Graph Sequence Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1511.05493. URL: https://arxiv.org/abs/1511.05493.

Li, Zhong et al. (June 2024). "Explainable Graph Neural Networks Under Fire". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2406.06417. URL: http://arxiv.org/abs/2406.06417.

Luo, Dongsheng et al. (Jan. 2020). "Parameterized Explainer for Graph Neural Network". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2011.04573. URL: https://arxiv.org/abs/2011.04573.

Martinkus, Karolis et al. (Jan. 2022). "Agent-based Graph Neural Networks". In: *arXiv (Cornell Univer-*

sity). DOI: 10.48550/arxiv.2206.11010. URL: https://arxiv.org/abs/2206.11010.

Oono, Kenta and Taiji Suzuki (Jan. 2019). "Graph Neural Networks Exponentially Lose Expressive Power for Node Classification". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1905.10947. URL: https://arxiv.org/abs/1905.10947.

Song, Chunyao et al. (Dec. 2020). "Similar but foreign: Link recommendation across communities". In: *Information Sciences* 552, pp. 142–166. ISSN: 0020-0255, 1872-6291. DOI: 10.1016/j.ins.2020.11.049. URL: https://doi.org/10.1016/j.ins.2020.11.049.

Thekumparampil, Kiran Koshy et al. (Jan. 2018). "Attention-based Graph Neural Network for Semi-supervised Learning". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1803.03735. URL: https://arxiv.org/abs/1803.03735.

Tola, Abdullah T., Jack Myrick, and Baris Coskunuzer (Oct. 2024). "PROXI: Challenging the GNNs for Link Prediction". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2410.01802. URL: http://arxiv.org/abs/2410.01802.

Veličković, Petar et al. (Jan. 2017). "Graph Attention Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1710.10903. URL: https://arxiv.org/abs/1710.10903.

Ying, Rex et al. (Jan. 2019). "GNNExplainer: Generating Explanations for Graph Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1903.03894. URL: https://arxiv.org/abs/1903.03894.

Zhang, Muhan and Yixin Chen (Jan. 2018). "Link Prediction Based on Graph Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.1802.09691. URL: https://arxiv.org/abs/1802.09691.

Zhao, Tianyi, Jian Kang, and Lu Cheng (June 2024). "Conformalized Link Prediction on Graph Neural Networks". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2406.18763. URL: http://arxiv.org/abs/2406.18763.

Zhao, Zicheng et al. (Jan. 2023). "Towards Few-shot Inductive Link Prediction on Knowledge Graphs: A Relational Anonymous Walk-guided Neural Process Approach". In: *arXiv (Cornell University)*. DOI: 10.48550/arxiv.2307.01204. URL: https://arxiv.org/abs/2307.01204.

Zheng, Yanping, Yi Lü, and Zhewei Wei (Dec. 2024). "A survey of dynamic graph neural networks". In: *Frontiers of Computer Science* 19 (6). ISSN: 2095-2228, 2095-2236. DOI: 10.1007/s11704-024-3853-2. URL: https://doi.org/10.1007/s11704-024-3853-2.

Zhou, Jie et al. (Jan. 2020). *Graph neural networks: A review of methods and applications*. DOI: 10.1016/j.aiopen.2021.01.001. URL: https://doi.org/10.1016/j.aiopen.2021.01.001.

Ziya, Fatima and Sanjay Kumar (Aug. 2024). "GSVAELP: integrating graphSAGE and variational autoencoder for link prediction". In: *Multimedia Tools and Applications*. ISSN: 1380-7501, 1573-7721. DOI: 10.1007/s11042-024-20123-z. URL: https://doi.org/10.1007/s11042-024-20123-z.